



UNIVERSITAS UDAYANA

MANUAL BOOK

MOVIELENS

**BANK PERKREDITAN
RAKYAT LESTARI**

Disusun Oleh :

Ida Bagus Gede Basudewa Weda

Dr. Ir. Ngurah Agus Sanjaya ER, S.Kom., M.Kom.

Cokorda Rai Adi Pramatha, ST.MMSI.,Ph.D

Ir. Gst. Ayu Vida Mastrika Giri, S.Kom. M.Cs.

DAFTAR ISI

I.	PENDAHULUAN	1
II.	PENGOPERASIAN APLIKASI	3
1.	Instalasi Aplikasi.....	3
1.1	Prasyarat Sistem.....	3
1.2	Instalasi dan Konfigurasi <i>Backend</i> (FastAPI).....	4
1.3	Instalasi dan Konfigurasi <i>Frontend</i> (Next.js)	5
1.4	Urutan Menjalankan Aplikasi	5
2.	Fitur Aplikasi	6
2.1	Registrasi Akun.....	6
2.2	<i>Login</i> dan <i>Logout</i>	6
2.3	Beranda dan Rekomendasi Personal	7
2.4	Rekomendasi Berdasarkan Genre	8
2.5	Daftar Film yang Sudah Dirating	9
2.6	Detail Film	9
2.7	Memberi Rating Film.....	10
2.8	Pembaruan Rekomendasi Secara <i>Real-time</i> (<i>Fold-in</i>).....	11
III.	POTONGAN PROGRAM.....	13
1.	<i>Minimum Library Requirements</i>	13
2.	Aktivasi <i>Virtual Environment</i>	13
3.	Memuat Model <i>Hybrid VAE + RSVD</i>	14
4.	Perhitungan Matriks Prediksi Awal	14
5.	<i>Endpoint Login</i> dan Verifikasi <i>Password</i>	15
6.	<i>Endpoint</i> Registrasi Pengguna Baru	16
7.	Menghasilkan Rekomendasi Personal	16
8.	Rekomendasi Berdasarkan Genre	17
9.	Menyimpan Rating dan Proses <i>Fold-in</i>	18
10.	Pemanggilan API dari <i>Frontend</i>	19
11.	Komponen <i>Shelf</i> Rekomendasi	19
12.	Modal Detail Film dan Fitur Rating.....	20

I. PENDAHULUAN

MovieMatch merupakan aplikasi sistem rekomendasi film berbasis web yang dikembangkan sebagai implementasi dari model *machine learning hybrid* Variational Autoencoder (VAE) dan Regularized Singular Value Decomposition (RSVD) yang telah dirancang, dilatih, dan dievaluasi pada penelitian tugas akhir ini. Aplikasi dibangun untuk menunjukkan bagaimana model prediksi rating yang dihasilkan dapat dimanfaatkan secara langsung dalam bentuk antarmuka yang interaktif, sehingga rekomendasi film tidak hanya tersaji sebagai angka pada tabel evaluasi, tetapi juga dapat dijelajahi layaknya sebuah *streaming platform*.

Model yang menjadi mesin utama aplikasi ini dilatih menggunakan dataset MovieLens 100K yang terdiri atas 943 pengguna, 1.682 film, dan 100.000 rating. Berdasarkan pola rating tersebut, model memprediksi nilai rating yang kemungkinan akan diberikan seorang pengguna terhadap film-film yang belum pernah ia tonton. Film dengan prediksi rating tertinggi kemudian ditampilkan sebagai rekomendasi. Pendekatan *hybrid* menggabungkan kekuatan VAE dalam menangkap representasi laten dari keseluruhan pola rating seorang pengguna dengan keunggulan RSVD dalam memodelkan kemiripan selera antarpengguna melalui faktorisasi matriks.

Secara arsitektur, aplikasi terdiri atas dua bagian yang berjalan secara terpisah. Bagian *backend* dikembangkan menggunakan Python dengan kerangka kerja FastAPI, yang bertugas memuat model, menghitung matriks prediksi, dan menyediakan layanan berupa API. Bagian *frontend* dikembangkan menggunakan Next.js sebagai antarmuka yang diakses pengguna melalui *browser*. Kedua bagian tersebut berkomunikasi melalui protokol HTTP, dengan *frontend* mengambil seluruh data dari *backend*.

Melalui aplikasi ini, pengguna dapat masuk sebagai salah satu pengguna bawaan dataset maupun mendaftarkan akun baru, menjelajahi rekomendasi film yang disusun secara personal maupun per genre, melihat detail sebuah film, serta memberikan rating. Salah satu fitur utama yang membedakan aplikasi ini dari sekadar penyajian hasil statis adalah kemampuan memperbarui rekomendasi secara langsung ketika pengguna memberikan rating baru, tanpa perlu melatih ulang model. Dengan demikian, aplikasi tidak hanya menyajikan hasil akhir penelitian, tetapi juga memperlihatkan bagaimana model bekerja secara dinamis dalam merespons masukan pengguna.

Perlu ditekankan bahwa MovieMatch dikembangkan sebagai demonstrasi lokal untuk keperluan tugas akhir, bukan sebagai produk publik. Seluruh proses dijalankan pada lingkungan lokal pengguna, dan mekanisme autentikasi yang digunakan bersifat sederhana sesuai kebutuhan demonstrasi.

II. PENGOPERASIAN APLIKASI

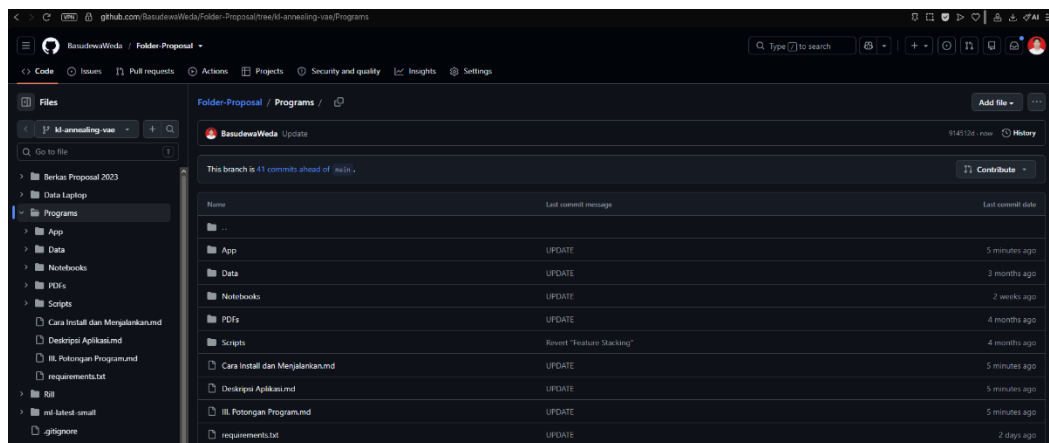
1. Instalasi Aplikasi

Aplikasi MovieMatch terdiri atas dua komponen yang harus disiapkan, yaitu *backend* dan *frontend*. Seluruh perintah pada bagian ini dijalankan melalui PowerShell atau Terminal dari dalam folder Programs, kecuali disebutkan lain. Karena *frontend* mengambil seluruh data dari *backend*, komponen *backend* harus dinyalakan terlebih dahulu.

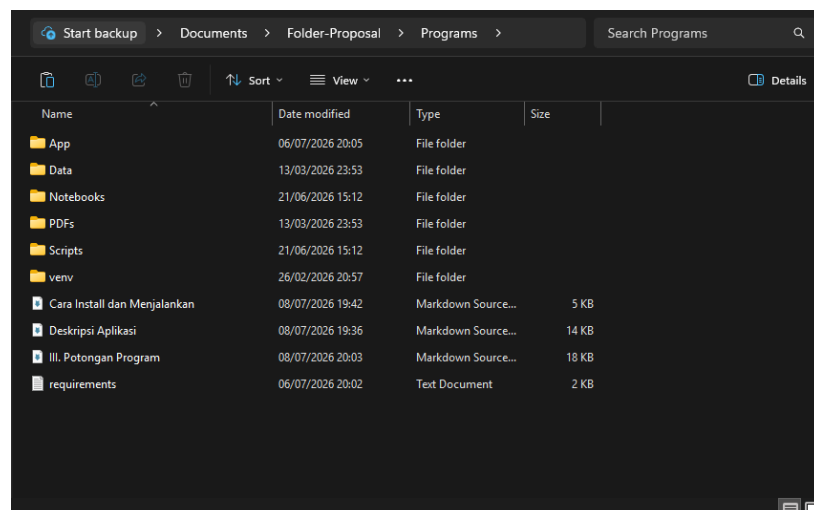
Program aplikasi secara lengkap dapat diakses melalui tautan GitHub berikut:

<https://github.com/BasudewaWeda/Folder-Proposal/tree/kl-annealing-vae-continue/Programs>

Melalui tautan tersebut, pengguna dapat mengunduh aplikasi ataupun melakukan *clone repository* menggunakan Git.



Gambar 1. Repository Program MovieMatch



Gambar 2. Struktur File Program MovieMatch

1.1 Prasyarat Sistem

Sebelum menjalankan aplikasi, pastikan perangkat telah terpasang perangkat lunak berikut:

- Python 3.11 untuk menjalankan *backend*.
- Node.js 18 untuk menjalankan *frontend*.

1.2 Instalasi dan Konfigurasi *Backend* (FastAPI)

Langkah pertama adalah menyiapkan lingkungan Python beserta seluruh pustaka yang dibutuhkan *backend*. Dari dalam folder Programs, jalankan perintah berikut untuk membuat dan mengaktifkan *virtual environment*:

```
# Membuat dan mengaktifkan virtual environment
python -m venv venv
.\venv\Scripts\Activate.ps1      # PowerShell
# atau: venv\Scripts\activate.bat # CMD

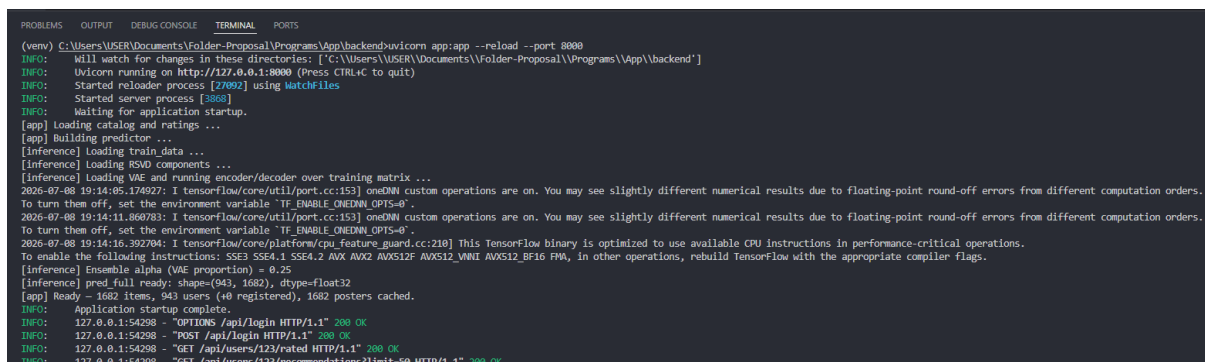
# Bila PowerShell menolak karena execution policy, jalankan sekali:
# Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

# Memasang seluruh dependency (agak lama karena memuat TensorFlow)
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Apabila *virtual environment* berhasil diaktifkan, akan muncul penanda (venv) di depan baris terminal. Setelah seluruh pustaka terpasang, jalankan *backend* dengan perintah:

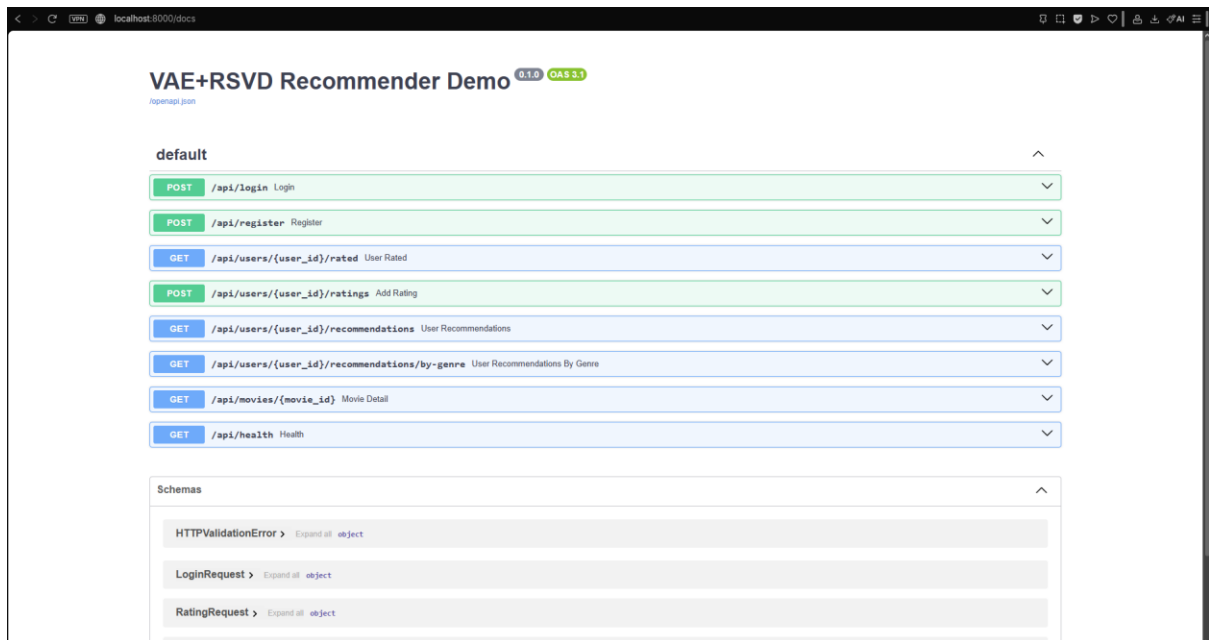
```
cd App\backend
uvicorn app:app --reload --port 8000
```

Layanan *backend* akan berjalan pada alamat `http://localhost:8000`, dengan dokumentasi API interaktif tersedia pada `http://localhost:8000/docs`. Proses pemuatan pertama membutuhkan waktu beberapa saat karena seluruh matriks prediksi dihitung terlebih dahulu. Biarkan terminal ini tetap terbuka selama aplikasi digunakan.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(venv) C:\Users\USER\Documents\Folder-Proposal\Programs\App\backend>uvicorn app:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\Users\USER\Documents\Folder-Proposal\Programs\App\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [27092] using WatchFiles
INFO: Started server process [3068]
INFO: Waiting for application startup.
[app] Loading catalog and ratings ...
[app] Building predictor ...
[app] Loading train data ...
[Inference] Loading RSV components ...
[Inference] Loading ME and running encoder/decoder over training matrix ...
2026-07-08 19:14:05.174927: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders.
To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-07-08 19:14:11.860783: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders.
To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-07-08 19:14:16.392784: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI AVX512_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
[Inference] Ensemble alpha (VAE proportion) = 0.25
[Inference] pred full ready: shape=(943, 1682), dtype=float32
[app] Ready - 1682 items, 943 users (+0 registered), 1682 posters cached.
INFO: Application startup complete.
INFO: 127.0.0.1:54298 - "OPTIONS /api/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:54298 - "POST /api/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:54298 - "GET /api/users/123/rated HTTP/1.1" 200 OK
INFO: 127.0.0.1:54298 - "GET /api/users/123/recommendations?limit=50 HTTP/1.1" 200 OK
```

Gambar 3. *Backend* MovieMatch Berhasil Dijalankan



Gambar 4. Dokumentasi API Interaktif (*Swagger UI*)

1.3 Instalasi dan Konfigurasi *Frontend* (Next.js)

Buka terminal baru (bukan terminal yang digunakan untuk menjalankan *backend*), lalu jalankan perintah berikut dari dalam folder Programs:

```
cd App\frontend
npm install      # cukup dijalankan sekali di awal
npm run dev
```

Setelah proses selesai, antarmuka aplikasi dapat diakses melalui `http://localhost:3000` pada *browser*.



Gambar 5. *Frontend* MovieMatch Berhasil Dijalankan

1.4 Urutan Menjalankan Aplikasi

Karena *frontend* bergantung pada *backend*, kedua komponen harus dinyalakan secara berurutan menggunakan dua terminal terpisah:

```
Terminal 1 -> cd App\backend -> aktifkan venv -> uvicorn app:app --reload --port 8000
Terminal 2 -> cd App\frontend -> npm run dev
```

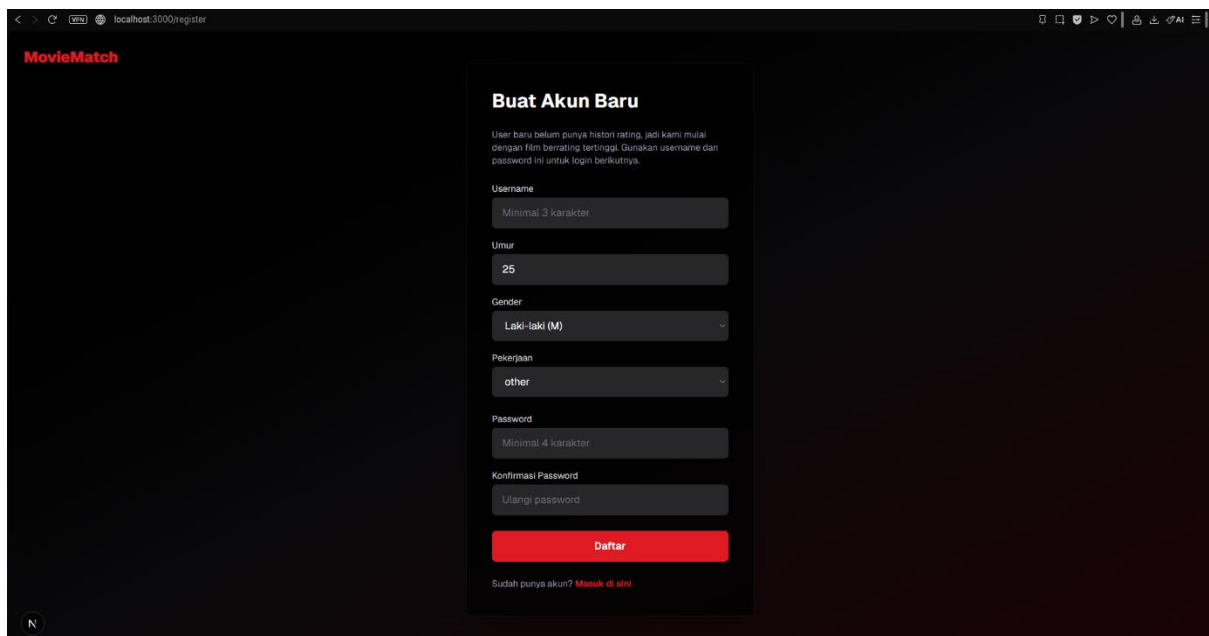
Setelah keduanya berjalan, buka `http://localhost:3000` di *browser* untuk mulai menggunakan aplikasi.

2. Fitur Aplikasi

2.1 Registrasi Akun

Pengguna yang belum memiliki akun dapat melakukan pendaftaran melalui halaman Daftar. Formulir pendaftaran meminta pengguna mengisi *username*, umur, *gender*, pekerjaan, *password*, dan konfirmasi *password*. Pilihan pekerjaan mengikuti kosakata MovieLens yang terdiri atas 21 pilihan, seperti administrator, artist, doctor, educator, engineer, programmer, dan student.

Sistem menerapkan sejumlah aturan validasi. Pada sisi *frontend*, *username* harus terdiri atas minimal tiga karakter, *password* minimal empat karakter, dan konfirmasi *password* harus cocok. Pada sisi *backend*, *username* harus unik dan tidak boleh menggunakan format `User_<angka>` karena format tersebut dicadangkan untuk pengguna bawaan dataset. Setelah pendaftaran berhasil, sistem memberikan ID pengguna secara otomatis mulai dari 944 di belakang layar, dan pengguna langsung masuk serta diarahkan ke beranda.

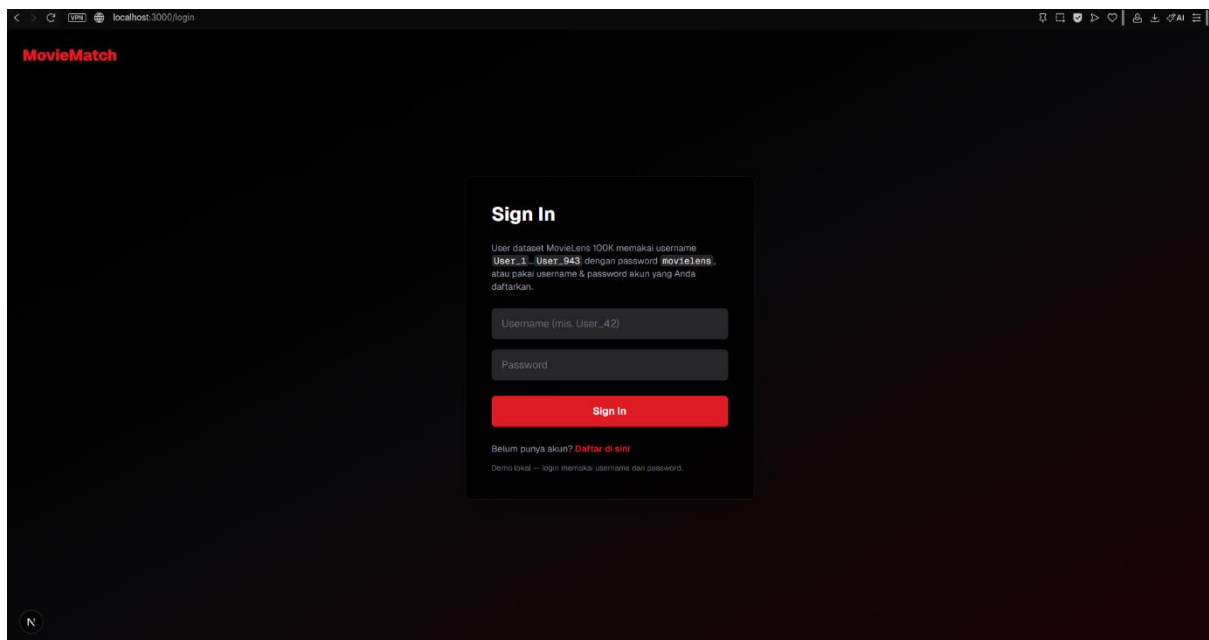


Gambar 6. Halaman Registrasi Akun

2.2 Login dan Logout

Aplikasi menggunakan mekanisme *login* berbasis *username* dan *password*. Terdapat dua jenis akun. Pertama, akun pengguna bawaan dataset dengan *username* User_1 sampai User_943 (misalnya User_42) yang seluruhnya menggunakan *password* movielens. Pengguna jenis ini telah memiliki histori rating dari dataset, sehingga rekomendasinya dihasilkan langsung oleh model *hybrid*. Kedua, akun baru hasil registrasi mandiri yang menggunakan *username* dan *password* pilihan pengguna sendiri.

Setelah berhasil masuk, sesi pengguna disimpan pada *localStorage browser* sehingga pengguna tetap dalam keadaan *login* meskipun halaman dimuat ulang. Apabila terjadi kesalahan, sistem menampilkan pesan yang jelas, seperti *Password* salah atau *Username* tidak ditemukan. Tombol *Logout* yang terletak di sudut kanan atas berfungsi menghapus sesi dan mengembalikan pengguna ke halaman *login*. Halaman yang memerlukan status masuk akan otomatis mengalihkan pengguna ke halaman *login* apabila sesi belum tersedia.

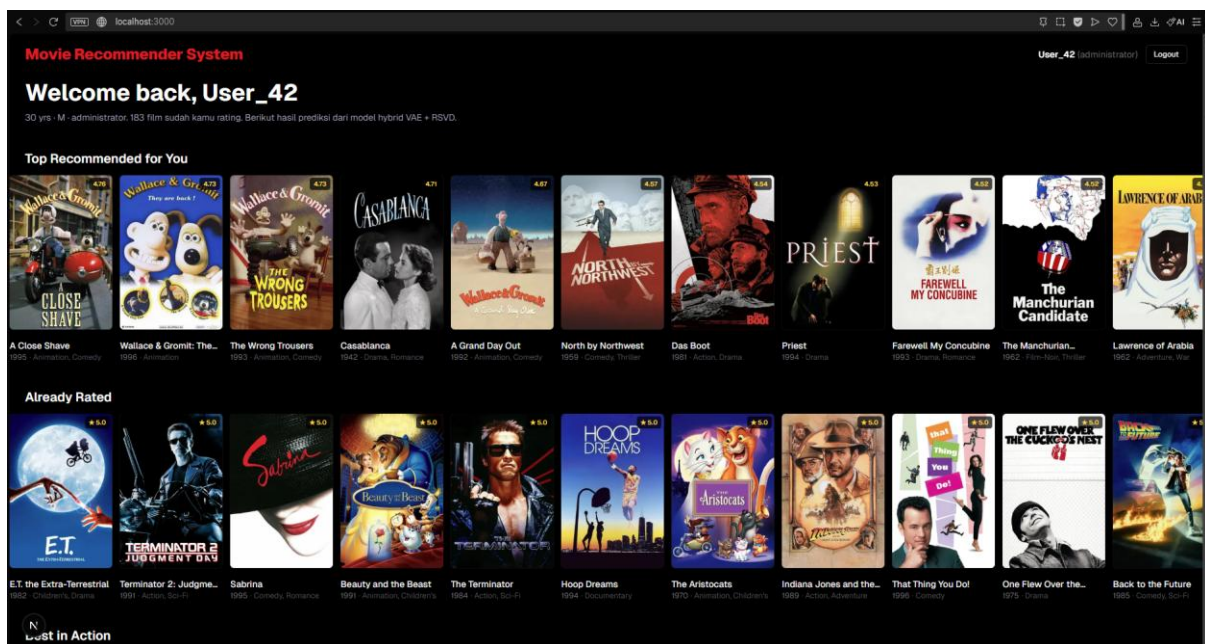


Gambar 7. Halaman *Login*

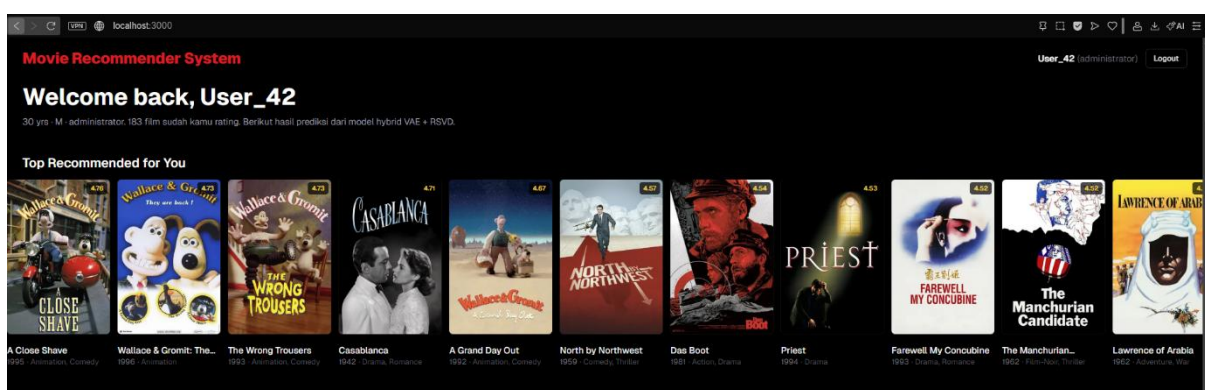
2.3 Beranda dan Rekomendasi Personal

Setelah berhasil masuk, pengguna diarahkan ke beranda yang menampilkan antarmuka bergaya *streaming platform*. Pada bagian atas terdapat *navbar* yang memuat nama aplikasi, *username* beserta pekerjaan pengguna, dan tombol *Logout*. Di bawahnya terdapat sapaan yang menampilkan nama pengguna, ringkasan demografi (umur, *gender*, dan pekerjaan), serta jumlah film yang telah dirating.

Bagian utama beranda tersusun atas beberapa *shelf*, yaitu deretan poster film yang dapat digeser secara horizontal. *Shelf* pertama berjudul Top Recommended for You berisi film dengan prediksi rating tertinggi bagi pengguna, yang dihasilkan oleh model *hybrid* VAE + RSVD. Film yang sudah pernah dirating pengguna dikecualikan dari daftar rekomendasi agar pengguna hanya disuguhi film yang belum ia tonton. Khusus untuk pengguna baru yang belum memiliki histori rating (*cold-start*), judul *shelf* berubah menjadi Film dengan Rating Tertinggi dan menampilkan film dengan rata-rata rating tertinggi dengan syarat minimal 50 vote, agar rekomendasi tidak bias terhadap film yang jarang dirating.



Gambar 8. Halaman Beranda MovieMatch

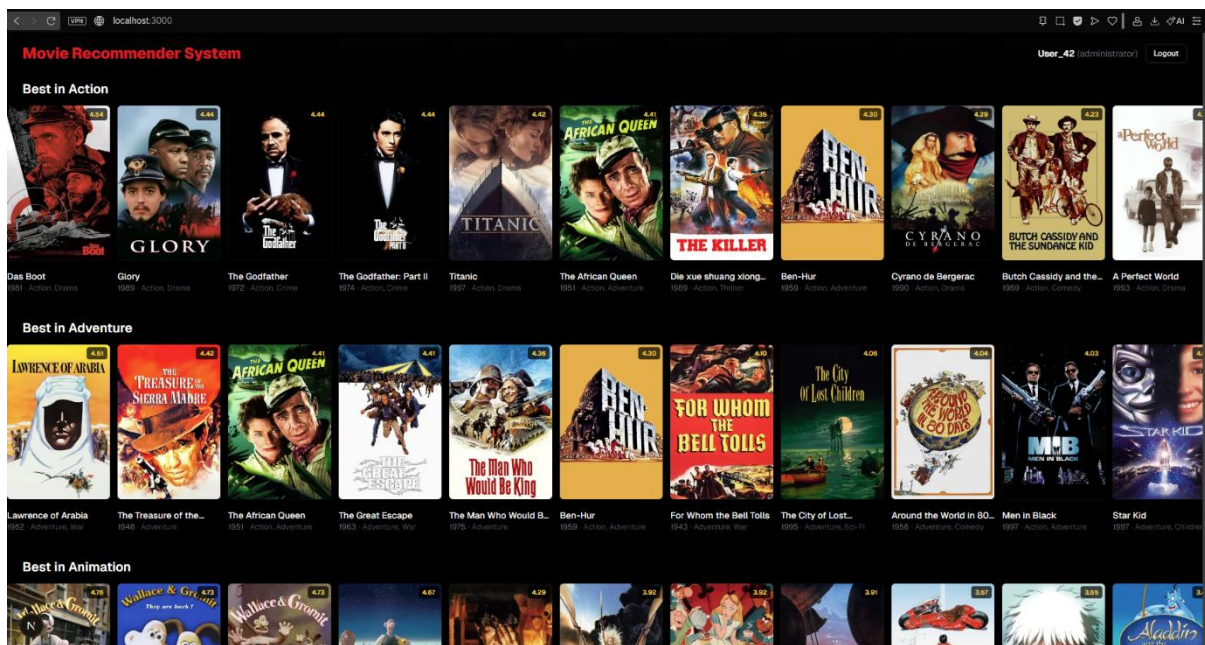


Gambar 9. *Shelf* Rekomendasi Personal

2.4 Rekomendasi Berdasarkan Genre

Selain rekomendasi personal, beranda juga menampilkan *shelf* tersendiri untuk setiap genre dengan judul Best in ... Setiap *shelf* genre berisi film terbaik menurut prediksi model pada genre

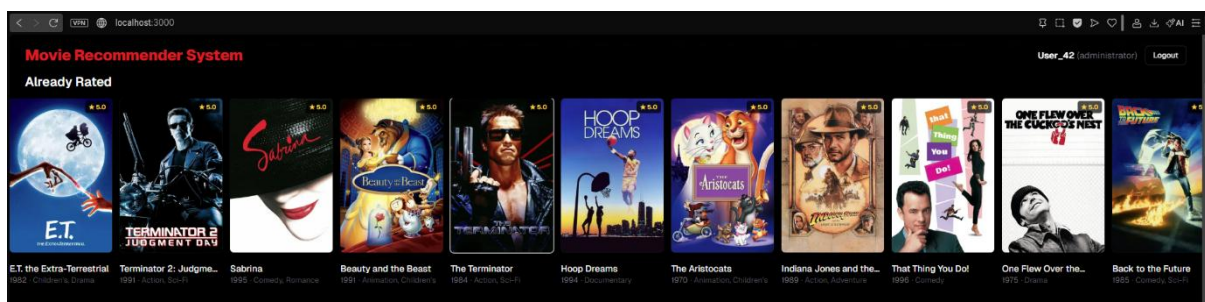
tersebut. Genre yang digunakan mengikuti 18 kategori MovieLens, yaitu Action, Adventure, Animation, Children's, Comedy, Crime, Documentary, Drama, Fantasy, Film-Noir, Horror, Musical, Mystery, Romance, Sci-Fi, Thriller, War, dan Western. Dengan pengelompokan ini, pengguna dapat menelusuri rekomendasi sesuai preferensi genre yang diminati.



Gambar 10. Shelf Rekomendasi Berdasarkan Genre

2.5 Daftar Film yang Sudah Dirating

Beranda turut menampilkan *shelf* Already Rated yang memuat seluruh film yang telah dirating pengguna, diurutkan dari rating tertinggi. *Shelf* ini menggabungkan rating asli dari dataset (untuk pengguna bawaan dataset) dengan rating yang diberikan langsung melalui aplikasi. Apabila terdapat perbedaan antara keduanya, rating yang diberikan melalui aplikasi diprioritaskan. Pada *shelf* ini, *badge* pada poster menampilkan rating bintang aktual (misalnya bintang 5.0), bukan prediksi.

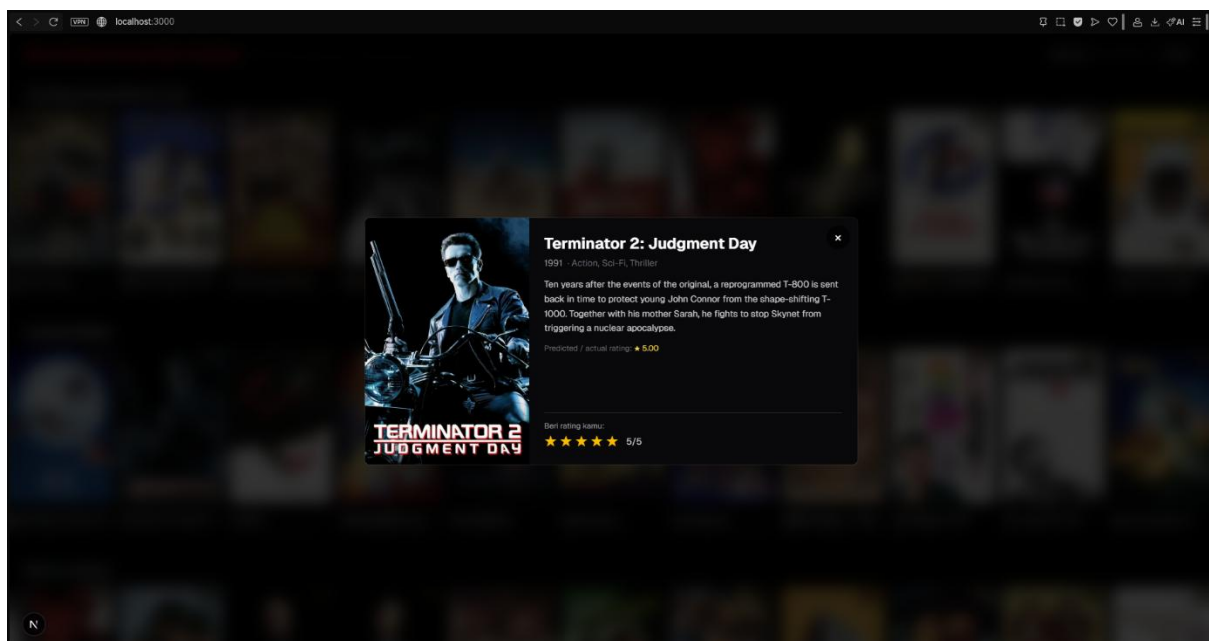


Gambar 11. Shelf Film yang Sudah Dirating

2.6 Detail Film

Setiap film ditampilkan sebagai kartu poster dengan rasio 2:3 yang memuat judul, tahun, dan maksimal dua genre. Poster diambil dari *cache* lokal; apabila poster suatu film tidak tersedia, judul film ditampilkan sebagai penggantinya. Ketika kursor diarahkan ke sebuah kartu, kartu tersebut sedikit membesar disertai efek *highlight*.

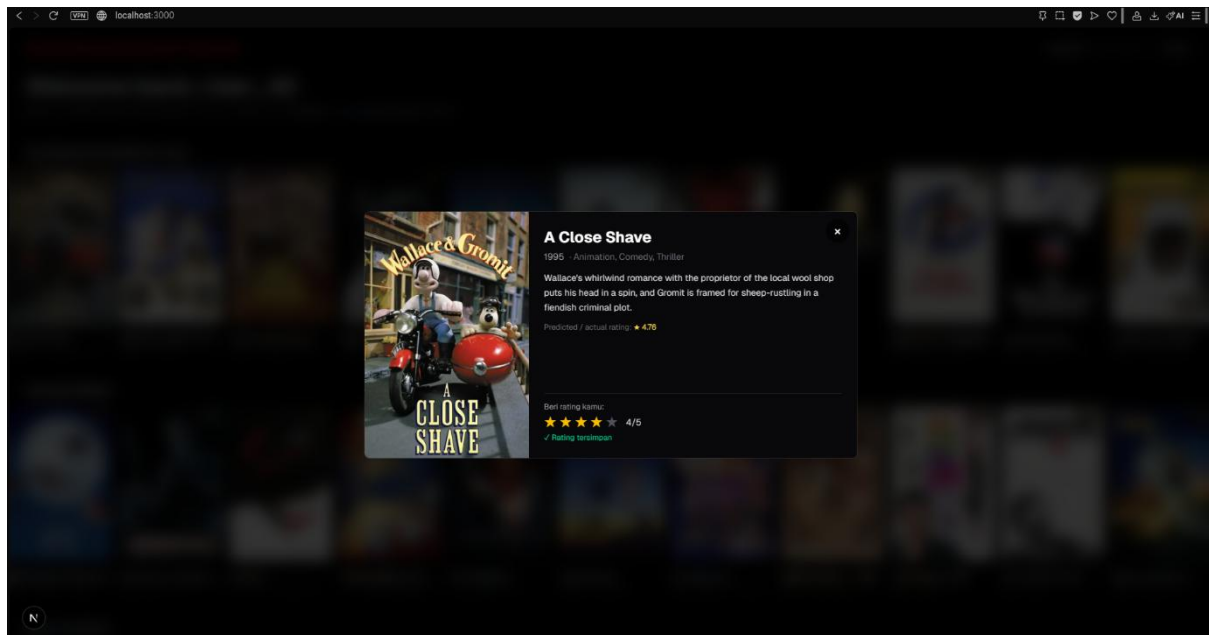
Ketika sebuah kartu diklik, aplikasi menampilkan modal detail film dengan animasi *fade* dan *zoom*. Modal ini memperlihatkan poster berukuran besar, judul, tahun, seluruh genre, sinopsis film, serta prediksi atau rating aktual. Modal dapat ditutup melalui tombol silang, dengan mengklik area gelap di luar modal, atau dengan menekan tombol Esc. Selama modal terbuka, halaman di belakangnya dikunci agar tidak ikut ter-*scroll*.



Gambar 12. Modal Detail Film

2.7 Memberi Rating Film

Di dalam modal detail film tersedia pilihan bintang 1 sampai 5 yang digunakan pengguna untuk memberikan rating. Saat kursor diarahkan pada bintang, aplikasi menampilkan pratinjau jumlah bintang yang akan dipilih. Setelah rating dipilih, nilai tersebut langsung dikirim dan disimpan ke *backend*, disertai umpan balik berupa keterangan Menyimpan... yang kemudian berubah menjadi Rating tersimpan apabila proses berhasil.

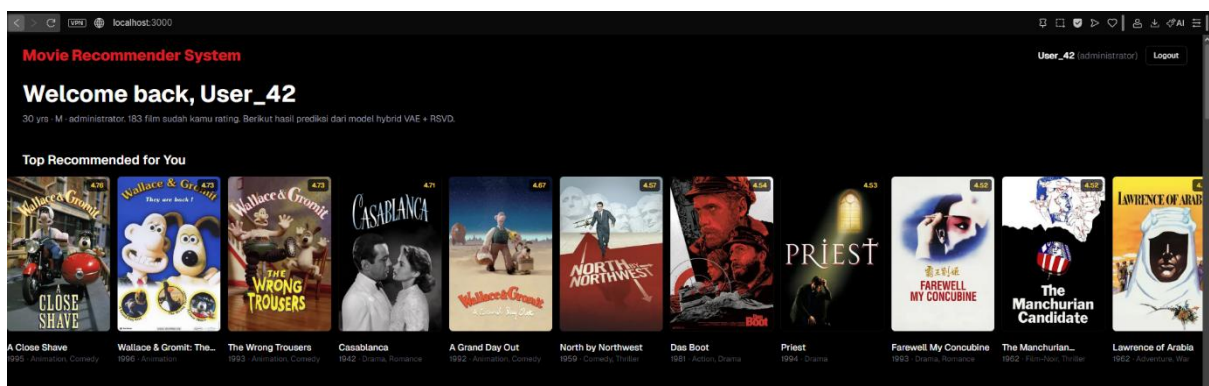


Gambar 13. Fitur Memberi Rating Film

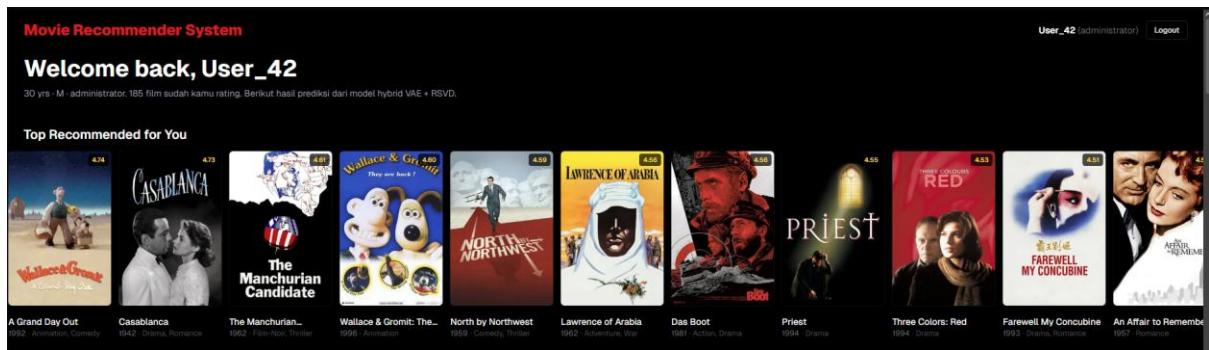
2.8 Pembaruan Rekomendasi Secara *Real-time (Fold-in)*

Fitur ini merupakan pembeda utama aplikasi dibandingkan sekadar menampilkan tabel prediksi statis. Ketika pengguna memberikan rating baru, *backend* melakukan proses *fold-in*, yaitu memasukkan rating baru tersebut ke dalam vektor rating pengguna dan menghitung ulang prediksi tanpa perlu melatih ulang model. Setelah modal ditutup, *shelf* diperbarui secara otomatis, film yang baru saja dirating berpindah ke *shelf* Already Rated, sementara daftar rekomendasi ikut menyesuaikan, semuanya berlangsung tanpa perlu memuat ulang halaman.

Proses *fold-in* berlaku baik untuk pengguna bawaan dataset maupun pengguna baru. Semakin banyak film yang dirating, rekomendasi yang dihasilkan akan semakin sesuai dengan selera pengguna.



Gambar 14. Rekomendasi Sebelum Pemberian Rating



Gambar 15. Perubahan Rekomendasi Setelah Pemberian Rating (*Fold-in*)

III. POTONGAN PROGRAM

Bagian ini menampilkan potongan kode inti dari aplikasi MovieMatch beserta penjelasan singkatnya. Kode dikelompokkan sesuai alur kerja aplikasi, mulai dari penyiapan lingkungan, pemuatan model, penyediaan layanan (API) di sisi *backend*, hingga antarmuka di sisi *frontend*.

1. *Minimum Library Requirements*

Aplikasi membutuhkan sejumlah pustaka (*library*) Python. Berikut pustaka utama beserta versinya (daftar lengkap tersedia pada berkas `requirements.txt`).

Berkas: `Programs/requirements.txt`

```
# Machine learning & komputasi numerik
tensorflow==2.20.0
keras==3.12.1
numpy==2.2.6
pandas==2.3.3
scikit-learn==1.7.2
scipy==1.15.3
optuna==4.7.0

# Web service (backend API)
fastapi==0.115.5
uvicorn==0.32.1
pydantic==2.13.4
requests==2.32.5

# Notebook & visualisasi
jupyter_client==8.8.0
ipykernel==7.2.0
matplotlib==3.10.8
tqdm==4.67.3
```

Seluruh pustaka dipasang sekaligus dengan perintah:

```
pip install -r requirements.txt
```

2. Aktivasi *Virtual Environment*

Sebelum menjalankan program, dibuat lingkungan virtual (*virtual environment*) agar seluruh pustaka terisolasi dari sistem. Perintah dijalankan dari folder `Programs`.

```
# Membuat virtual environment bernama "venv"
python -m venv venv

# Mengaktifkan virtual environment (PowerShell)
.\venv\Scripts\Activate.ps1

# Bila PowerShell menolak karena execution policy, jalankan sekali:
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

# Memasang seluruh dependency
```

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Jika berhasil, akan muncul penanda (venv) di depan baris terminal.

3. Memuat Model *Hybrid* VAE + RSVD

Saat *backend* dinyalakan, kedua model dimuat dari folder Data. Komponen RSVD berupa berkas NumPy (.npz), sedangkan VAE dimuat dari bobot terlatih (.weights.h5) menggunakan arsitektur yang sama seperti saat pelatihan.

Berkas: App/backend/inference.py

```
def _load_rsvd_components():
    mu = np.load(RSVD_DIR / "final_mu.npz")
    b_u = np.load(RSVD_DIR / "final_b_u.npz")
    b_i = np.load(RSVD_DIR / "final_b_i.npz")
    U = np.load(RSVD_DIR / "best_U.npz")
    Sigma = np.load(RSVD_DIR / "best_Sigma.npz")
    V = np.load(RSVD_DIR / "best_V.npz")
    return mu, b_u, b_i, U, Sigma, V

def _load_vae(train_data: np.ndarray):
    from model import Encoder, Decoder, VAE

    with open(VAE_PARAMS_PATH) as f:
        params = json.load(f)["best_params"]

    num_items = train_data.shape[1]
    encoder = Encoder(
        hidden_dims = params["hidden_dims"],
        latent_dim = params["latent_dim"],
        dropout_rate = params["dropout_rate"],
    )
    decoder = Decoder(hidden_dims=params["hidden_dims"][:-1], output_dim=num_items)
    vae = VAE(encoder, decoder, beta=params["beta"])
    # Bangun variabel model dulu sebelum memuat bobot
    _ = vae(train_data[:1])
    vae.load_weights(str(VAE_WEIGHTS_PATH))
    return vae
```

4. Perhitungan Matriks Prediksi Awal

Setelah kedua model dimuat, seluruh matriks prediksi rating berukuran 943 x 1682 dihitung satu kali di awal. Prediksi akhir merupakan gabungan berbobot (*ensemble*) dari VAE dan RSVD. Dengan cara ini, setiap permintaan rekomendasi cukup membaca satu baris matriks (cepat).

Berkas: App/backend/inference.py

```
def build_predictor() -> Predictor:
    # 1) Muat data latih dan komponen RSVD, lalu susun prediksi RSVD
    train_data = np.load(TRAIN_DATA_PATH).astype(np.float32)
```



```

mu, b_u, b_i, U, Sigma, V = _load_rsvd_components()
pred_rsvd_norm = float(mu) + b_u[:, None] + b_i[None, :] + U @ Sigma @ V.T
pred_rsvd = np.clip(pred_rsvd_norm * MAX_RATING, 1.0, MAX_RATING).astype(np.float32)

# 2) Jalankan encoder -> decoder VAE atas seluruh matriks rating
vae = _load_vae(train_data)
z_mean, _ = vae.encoder.predict(train_data, verbose=0)
pred_vae_norm = vae.decoder.predict(z_mean, verbose=0)
pred_vae = np.clip(pred_vae_norm * MAX_RATING, 1.0, MAX_RATING).astype(np.float32)

# 3) Gabungkan kedua prediksi dengan bobot alpha (hasil penyetelan)
alpha = _latest_eval_alpha()
pred_full = np.clip(
    alpha * pred_vae + (1.0 - alpha) * pred_rsvd, 1.0, MAX_RATING
).astype(np.float32)

return Predictor(pred_full=pred_full, pred_rsvd=pred_rsvd,
                 train_data=train_data, alpha=alpha, vae=vae, ...)

```

5. Endpoint Login dan Verifikasi Password

Endpoint POST /api/login menerima *username* dan *password*. Pengguna hasil registrasi diverifikasi terhadap *hash password* tersimpan, sedangkan pengguna bawaan dataset (User_1-User_943) memakai *password* default movielens.

Berkas: App/backend/app.py

```

@app.post("/api/login")
def login(req: LoginRequest):
    username = req.username.strip()

    # 1) Pengguna hasil registrasi memiliki username bebas
    rec = _new_users().by_username(username)
    if rec is not None:
        if not _new_users().check_password(rec["user_id"], req.password):
            raise HTTPException(status_code=401, detail="Password salah")
        return {**rec, "is_new": True}

    # 2) Pengguna dataset login sebagai "User_<id>" + password default
    user_id = _dataset_username_id(username)
    if user_id is not None and user_id in _catalog().users.index:
        if req.password != DEFAULT_PASSWORD:
            raise HTTPException(status_code=401, detail="Password salah")
        info = _catalog().users.loc[user_id]
        return {
            "user_id": user_id, "username": f"User_{user_id}",
            "age": int(info["age"]), "gender": str(info["gender"]),
            "occupation": str(info["occupation"]), "is_new": False,
        }

    raise HTTPException(status_code=401, detail="Username tidak ditemukan")

```

Verifikasi *password* memakai algoritma PBKDF2-SHA256 dengan *salt*. *Password* tidak pernah disimpan sebagai teks polos.

Berkas: App/backend/users_store.py

```
def verify_password(password: str, stored: Optional[str]) -> bool:
    """Mencocokkan password dengan hash yang tersimpan."""
    if not stored:
        return False
    try:
        algo, iters, salt, expected = stored.split("$")
        if algo != "pbkdf2_sha256":
            return False
        digest = hashlib.pbkdf2_hmac(
            "sha256", password.encode("utf-8"), bytes.fromhex(salt), int(iters)
        )
    except (ValueError, AttributeError):
        return False
    return secrets.compare_digest(digest.hex(), expected)
```

6. Endpoint Registrasi Pengguna Baru

Endpoint POST `/api/register` membuat pengguna baru. *Backend* memvalidasi *username* (unik, minimal 3 karakter, bukan format `User_<angka>`) dan *password* (minimal 4 karakter), lalu menyimpan datanya.

Berkas: `App/backend/app.py`

```
@app.post("/api/register")
def register(req: RegisterRequest):
    username = req.username.strip()
    if len(username) < 3:
        raise HTTPException(status_code=400, detail="Username minimal 3 karakter")
    if _dataset_username_id(username) is not None:
        raise HTTPException(status_code=400,
            detail='Username dengan format "User_<angka>" dipakai user dataset')
    if _new_users().username_taken(username):
        raise HTTPException(status_code=409, detail="Username sudah dipakai")
    if not req.password or len(req.password) < 4:
        raise HTTPException(status_code=400, detail="Password minimal 4 karakter")
    info = _new_users().register(
        username=username, age=req.age, gender=req.gender,
        occupation=req.occupation, password=req.password,
    )
    return {**info, "is_new": True}
```

Password diubah menjadi *hash* bersalt sebelum disimpan ke berkas `new_users.json`.

Berkas: `App/backend/users_store.py`

```
def hash_password(password: str) -> str:
    """Menghasilkan hash PBKDF2-SHA256 bersalt: 'pbkdf2_sha256$iters$salt$hash'."""
    salt = secrets.token_hex(16)
    digest = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), bytes.fromhex(salt), _PBKDF2_ITERATIONS
    )
    return f"pbkdf2_sha256${_PBKDF2_ITERATIONS}${salt}${digest.hex()}"
```

7. Menghasilkan Rekomendasi Personal

Endpoint GET /api/users/{id}/recommendations mengambil skor prediksi pengguna, mengurutkannya dari tertinggi, lalu mengembalikan film-film teratas (film yang sudah dirating dikecualikan).

Berkas: App/backend/app.py

```
@app.get("/api/users/{user_id}/recommendations")
def user_recommendations(user_id: int, limit: int = 50):
    _ensure_user(user_id)
    scores = _scores_excluding_rated(user_id)
    top = np.argsort(scores)[::-1][:limit]
    return [
        _catalog().movie_card(int(idx + 1), rating=float(scores[idx]))
        for idx in top
        if np.isfinite(scores[idx])
    ]
```

Fungsi `_scores_excluding_rated` menentukan sumber skor: matriks *hybrid* untuk pengguna dataset, skor rata-rata (*cold-start*) untuk pengguna baru, atau hasil *fold-in* bila pengguna sudah memberi rating lewat aplikasi. Film yang sudah dirating diberi nilai negatif tak hingga agar tidak muncul kembali.

```
def _scores_excluding_rated(user_id: int) -> np.ndarray:
    app_ratings = _user_ratings().get_user(user_id)
    if app_ratings:
        # ada rating dari aplikasi
        vec = _user_rating_vector_norm(user_id)
        vae_pred = _predictor().predict_vae_foldin(vec)
        rsvd_pred = _predictor().predict_rsvd_foldin(vec)
        a = _predictor().alpha
        scores = np.clip(a * vae_pred + (1.0 - a) * rsvd_pred, 1.0, MAX_RATING)
    elif _is_new_user(user_id):
        # pengguna baru: cold-start
        scores = _avg_scores().copy()
    else:
        # pengguna dataset: matriks hybrid
        scores = _predictor().predictions_for_user(user_id).copy()

    scores = np.array(scores, dtype=np.float32, copy=True)
    scores[_excluded_mask(user_id)] = -np.inf
    # buang film yang sudah dirating
    return scores
```

8. Rekomendasi Berdasarkan Genre

Endpoint GET /api/users/{id}/recommendations/by-genre menyaring skor pengguna per genre, lalu mengembalikan film terbaik untuk masing-masing genre.

Berkas: App/backend/app.py

```
@app.get("/api/users/{user_id}/recommendations/by-genre")
def user_recommendations_by_genre(user_id: int, limit: int = 20):
    _ensure_user(user_id)
    scores = _scores_excluding_rated(user_id)
    out: dict[str, list[dict]] = {}
    for genre in SHELF_GENRES:
        mask = _catalog().genre_mask[genre]
        masked_scores = np.where(mask, scores, -np.inf)
        # film bergenre ini
        # sisanya diabaikan
```

```

top = np.argsort(masked_scores)[:limit]
cards = [
    _catalog().movie_card(int(idx + 1), rating=float(scores[idx]))
    for idx in top
    if np.isfinite(scores[idx])
]
if cards:
    out[genre] = cards
return out

```

9. Menyimpan Rating dan Proses *Fold-in*

Endpoint POST `/api/users/{id}/ratings` menyimpan rating yang diberikan pengguna lewat aplikasi. Rating ini kemudian ikut diperhitungkan saat menghitung rekomendasi melalui proses *fold-in*.

Berkas: App/backend/app.py

```

@app.post("/api/users/{user_id}/ratings")
def add_rating(user_id: int, req: RatingRequest):
    _ensure_user(user_id)
    if req.item_id not in _catalog().items.index:
        raise HTTPException(status_code=404, detail=f"Movie {req.item_id} tidak ditemukan")
    if not (1.0 <= req.rating <= MAX_RATING):
        raise HTTPException(status_code=400, detail="Rating harus antara 1 dan 5")
    _user_ratings().set(user_id, req.item_id, req.rating)
    return _catalog().movie_card(req.item_id, rating=req.rating)

```

Fold-in memungkinkan rating baru langsung memengaruhi prediksi tanpa melatih ulang model. Pada VAE, vektor rating pengguna cukup dilewatkan sekali melalui *encoder* dan *decoder*. Pada RSVD, vektor laten pengguna diestimasi ulang lewat regresi *ridge* atas faktor film yang sudah tetap.

Berkas: App/backend/inference.py

```

def predict_vae_foldin(self, rating_vec_norm: np.ndarray) -> np.ndarray:
    """Prediksi VAE untuk vektor rating apa pun (satu lintasan encoder->decoder)."""
    import tensorflow as tf
    x = tf.constant(rating_vec_norm[None, :], dtype=tf.float32)
    z_mean, _ = self.vae.encoder(x, training=False)
    pred_norm = self.vae.decoder(z_mean, training=False)
    pred = np.asarray(pred_norm)[0] * MAX_RATING
    return np.clip(pred, 1.0, MAX_RATING).astype(np.float32)

def predict_rsvd_foldin(self, rating_vec_norm: np.ndarray) -> np.ndarray:
    """Prediksi RSVD via fold-in: estimasi bias b_u dan vektor laten U[u]."""
    rated = np.where(rating_vec_norm > 0)[0]
    if rated.size == 0:
        # belum ada rating -> baseline bias
        base = self.rsvd_mu + self.rsvd_bi
        return np.clip(base * MAX_RATING, 1.0, MAX_RATING).astype(np.float32)

    X = self.rsvd_item_factors[rated]
    y = rating_vec_norm[rated] - self.rsvd_mu - self.rsvd_bi[rated]

```

```

Xa = np.concatenate([np.ones((rated.size, 1), dtype=X.dtype), X], axis=1)
reg = np.eye(Xa.shape[1], dtype=X.dtype)
reg[0, 0] = 0.0 # intercept tidak diregularisasi
w = np.linalg.solve(Xa.T @ Xa + self.rsvd_foldin_lambda * reg, Xa.T @ y)
b_u, u_vec = w[0], w[1:]
pred_norm = self.rsvd_mu + b_u + self.rsvd_bi + self.rsvd_item_factors @ u_vec
return np.clip(pred_norm * MAX_RATING, 1.0, MAX_RATING).astype(np.float32)

```

10. Pemanggilan API dari *Frontend*

Di sisi *frontend*, seluruh komunikasi dengan *backend* dikumpulkan dalam satu berkas. Setiap fungsi mengirim permintaan HTTP dan mengembalikan data JSON.

Berkas: App/frontend/src/lib/api.ts

```

export const API_BASE = "http://localhost:8000";

// Login dengan username + password
export async function login(username: string, password: string): Promise<LoginResponse>
{
  const res = await fetch(`${API_BASE}/api/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password }),
  });
  if (!res.ok) {
    const detail = await res.json().catch(() => ({ detail: res.statusText }));
    throw new Error(detail?.detail ?? `Login gagal: ${res.status}`);
  }
  return res.json();
}

// Mengambil rekomendasi personal
export function fetchRecommendations(userId: number, limit = 50) {
  return getJson<MovieCard[]>(`/api/users/${userId}/recommendations?limit=${limit}`);
}

// Mengirim rating baru
export async function rateMovie(userId: number, itemId: number, rating: number):
Promise<MovieCard> {
  const res = await fetch(`${API_BASE}/api/users/${userId}/ratings`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ item_id: itemId, rating }),
  });
  if (!res.ok) {
    const detail = await res.json().catch(() => ({ detail: res.statusText }));
    throw new Error(detail?.detail ?? `Gagal menyimpan rating: ${res.status}`);
  }
  return res.json();
}

```

11. Komponen *Shelf* Rekomendasi

Komponen *Shelf* menampilkan satu deret film (poster) yang dapat digeser horizontal, lengkap dengan tombol panah navigasi. Setiap film dirender melalui komponen *MovieCard*.

Berkas: App/frontend/src/app/components/Shelf.tsx

```

export default function Shelf({ title, movies, showRating, userId, onRated }: Props) {
  const scrollerRef = useRef<HTMLDivElement>(null);

  // Geser satu layar penuh ke kiri/kanan
  function scrollBy(direction: 1 | -1) {
    const el = scrollerRef.current;
    if (!el) return;
    el.scrollBy({ left: direction * el.clientWidth * 0.9, behavior: "smooth" });
  }

  return (
    <section className="group/shelf relative py-4">
      <h2 className="mb-3 px-6 text-xl font-semibold text-white">{title}</h2>

      <button type="button" aria-label="Scroll left" onClick={() => scrollBy(-1)}></button>
      <button type="button" aria-label="Scroll right" onClick={() => scrollBy(1)}></button>

      <div ref={scrollerRef} className="shelf-scroll flex snap-x gap-3 overflow-x-auto px-6">
        {movies.map((m) => (
          <MovieCard
            key={m.movie_id}
            movie={m}
            showRating={showRating}
            userId={userId}
            onRated={onRated}
          />
        ))}
      </div>
    </section>
  );
}

```

12. Modal Detail Film dan Fitur Rating

Komponen `MovieModal` menampilkan detail film (poster, sinopsis, genre) dan menyediakan pilihan bintang 1-5. Ketika bintang dipilih, rating dikirim ke *backend* melalui fungsi `rateMovie`, lalu daftar rekomendasi diperbarui saat modal ditutup.

Berkas: `App/frontend/src/app/components/MovieModal.tsx`

```

// Mengirim rating ke backend saat bintang dipilih
async function submitRating(star: number) {
  if (userId == null || saving) return;
  setSaving(true);
  setRateError(null);
  try {
    await rateMovie(userId, movie.movie_id, star);
    setRating(star);
    setSaved(true);
    ratedDuringSession.current = true;
  } catch (err) {
    setRateError(err instanceof Error ? err.message : "Gagal menyimpan rating");
  } finally {
    setSaving(false);
  }
}

```

Berikut potongan antarmuka pilihan bintang 1-5 di dalam modal:

```
{/* Pilihan bintang 1-5 di dalam modal */}
<div className="flex items-center gap-1">
  {[1, 2, 3, 4, 5].map((star) => {
    const active = (hover ?? rating ?? 0) >= star;
    return (
      <button
        key={star}
        type="button"
        disabled={saving}
        onMouseEnter={() => setHover(star)}
        onMouseLeave={() => setHover(null)}
        onClick={() => submitRating(star)}
        className={active ? "text-amber-400" : "text-zinc-600"}
      >
        ★
      </button>
    );
  )}
  {rating != null && <span className="ml-2 text-sm text-zinc-300">{rating}/5</span>}
</div>
```

Setelah modal ditutup, aplikasi memanggil ulang data sehingga film yang baru dirating berpindah ke daftar Already Rated dan rekomendasi ikut menyesuaikan tanpa memuat ulang halaman.

Berkas: App/frontend/src/app/components/MovieModal.tsx

```
function handleClose() {
  setVisible(false);
  window.setTimeout(() => {
    onClose();
    // Perbarui shelf hanya bila ada rating baru selama modal terbuka
    if (ratedDuringSession.current) onRated?();
  }, ANIM_MS);
}
```